矩阵快速幂
高斯消元
线性基
gcd,exgcd
中国剩余定理（CRT,exCRT）
Lucas定理
exLucas定理
数论分块
线性求逆元
组合数
三维前缀和
线性筛
欧拉函数
求一个数的最大质因子(1e18)
卡特兰数
快速傅里叶变换(FFT)
快速数论变换(NTT)
快速沃尔什变换(FWT)
多项式模板
取模类(int)

数学

矩阵快速幂

```
struct mat
{
    std::vector<std::vector<int>>a;
    mat(std::vector<std::vector<int>> &arr):a(arr){}
    mat operator*(const mat &tp)
    {
        auto &b = tp.a;
        int n = a.size(),m = b[0].size(),t = b.size();
        std::vector arr(n,std::vector<int>(m));
        for(int i = 0;i < n;i++)
            for(int j = 0;j < m;j++)
                for(int k = 0;k < t;k++)
                    arr[i][j] += a[i][k] * b[k][j];
        return mat(arr);
    }
    mat qmi(mat a,int b)
    {
        int n = a.a.size();
        std::vector arr(n,std::vector<int>(n));
        for(int i = 0;i < n;i++)arr[i][i] = 1;
        mat res(arr);
        while(b)
        {
            if(b & 1)res = res * a;
```

```
            a = a * a;
            b >>= 1;
        }
        return res;
    }
};

```

高斯消元

```
int gauss(std::vector<std::vector<int>> &arr)//为每一行找一个主元
{
    int lr = arr.size();
    int lc = arr[0].size();
    int row = 0;
    for(int col = 0;col < lr && row < lr;col++)
    {
        int tpr = row;
        for(int i = row;i < lr;i++)
            if(std::fabs(arr[tpr][col]) < std::fabs(arr[i][col]))tpr = i;
        if(std::fabs(arr[tpr][col]) < eps)continue;
        if(tpr != row)
            std::swap(arr[tpr],arr[row]);
        double w = arr[row][col];
        for(int j = col;j < lc;j++)arr[row][j] /= w;
        for(int i = 0;i < lr;i++)
        {
            if(i == row)continue;
            w = arr[i][col];
            for(int j = col;j < lc;j++)
                arr[i][j] -= w * arr[row][j];
        }
        row++;
    }
    return row;
}
void solve()
{
    int n;std::cin >> n;
    std::vector a(n,std::vector<double>(n + 1));
    for(int i = 0;i < n;i++)
        for(int j = 0;j <= n;j++)std::cin >> a[i][j];
    int rk = gauss(a);
    if(rk < n)std::cout << "No Solution\n";
    else
        for(int i = 0;i < n;i++)
            std::cout << std::fixed << std::setprecision(2) << a[i][n] << "\n";
}
```

线性基

```

```

```cpp
struct LBasis
{
    static constexpr int LOG = 62;
    int p[LOG + 1],rk;
    LBasis(int n = LOG + 1)
    {
        rk = 0;
        std::memset(p,0,sizeof(int) * n);
    }

    bool insert(long long x,int op = 1)//op = 1,插入
    {
        for(int i = LOG;i >= 0;i--)
        {
            if (!(x >> i & 1)) continue;
            if (!p[i])
            {
                if(op)
                {
                    p[i] = x;
                    rk++;
                }
                return true;
            }
            x ^= p[i];
        }
        return false;
    }
    int Qmx(int x = 0)
    {
        int ans = x;
        for(int i = LOG;i >= 0;i--)
            ans = std::max(ans,ans ^ p[i]);
        return ans;
    }
};
```

gcd, exgcd

```cpp
int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
int exgcd(int a, int b, int &x, int &y)
{
    if (!b)
    {
        x = 1;
        y = 0;
        return a;
    }
    int d = exgcd(b, a % b, x, y);
    int t = x;
    x = y;
    y = t - (a / b) * y;
```

```
    return d;
}
```

中国剩余定理（CRT, exCRT）

```
namespace CRT
{
    int exgcd(int a, int b, int &x, int &y)
    {
        if (!b)
        {
            x = 1;
            y = 0;
            return a;
        }
        int d = exgcd(b, a % b, x, y);
        int t = x;
        x = y;
        y = t - (a / b) * y;
        return d;
    }
    //返回通解 x === ai mod mi [0,n) m 两两互质
    pii CRT(int n,std::vector<int> &a,std::vector<int> &m)
    {
        int M = 1;
        for(int i = 0;i < n;i++)M = M * m[i];
        i128 ans = 0;
        for(int i = 0;i < n;i++)
        {
            int w = M / m[i];
            int inv,y;inv = y = 0; // The inv of w
            exgcd(w,m[i],inv,y);
            inv = (inv % m[i] + m[i]) % m[i];
            ans += i128(a[i]) * w % M * inv % M;
            ans %= M;
        }
        return {ans,M};
    }
    //返回通解 x === ai mod mi [0,n) m 两两不一定互质,{-1,-1} 为无解
    pii exCRT(int n,std::vector<int> &a,std::vector<int> &m)
    {
        int a1 = a[0],m1 = m[0];
        for(int i = 1;i < n;i++)
        {
            int x,y;
            int g = exgcd(m1,m[i],x,y);//gcd(m1,m[i])
            if((a[i] - a1) % g != 0)return {-1,-1};
            i128 P = (i128)x * ((a[i] - a1) / g);//特解，没有限制正负

            int M = m1 / g * m[i];//新的 m1

            a1 = (P * m1 % M + a1) % M;//新的 a1 = m1 * x + a1
```

```
            a1 = (a1 % M + M) % M;
            m1 = M;
        }
        return {a1,m1};
    }
};
```

Lucas定理

```
struct Lucas
{
    int p;//p <= 1e7 && isprime;

    std::vector<int>fa,infa;
    Lucas(int p):p(p),fa(p),infa(p)
    {
        fa[0] = 1;
        for(int i = 1;i < p;i++)fa[i] = fa[i - 1] * i % p;
        infa[p - 1] = qmi(fa[p - 1],p - 2);
        for(int i = p - 1;i >= 1;i--)infa[i - 1] = infa[i] * i % p;
    }
    int qmi(int a,int b)
    {
        int res = 1;
        while(b)
        {
            if(b & 1)res = res * a % p;
            a = a * a % p;
            b >>= 1;
        }
        return res;
    }
    int C(int n,int m)
    {
        if(n < m)return 0;
        return fa[n] * infa[m] % p * infa[n - m] % p;
    }
    int lucas(int n,int m)
    {
        if(n < p && m < p)return C(n,m);
        return C(n % p,m % p) * lucas(n / p,m / p) % p;
    }
};
Lucas lucas(p);
lucas.lucas(n,m);
```

exLucas定理

```cpp
struct exLucas
{
    struct node
    {
        int p,k,q;
        std::vector<int>c,s,f,f_inv;
        bool fl;
        int qmi(int a,int b)
        {
            int res = 1;
            while(b)
            {
                if(b & 1)res = res * a % q;
                a = a * a % q;
                b >>= 1;
            }
            return res;
        }
        node(int p,int k,int q,int n) : p(p),k(k),q(q),s(q),fl(0)
        {
            s[0] = 1;
            for(int i = 1;i < q;i++)
                s[i] = s[i - 1] * (i % p ? i : 1) % q;//分块，前 i 项积
            if(n <= N)
            {
                fl = 1;
                c.resize(n + 1);
                f.resize(n + 1);
                f_inv.resize(n + 1);
                f[0] = 1;
                for(int i = 1;i <= n;i++)
                {
                    c[i] = c[i - 1];
                    int cnt = i;
                    while(cnt % p == 0)c[i]++,cnt /= p;
                    f[i] = f[i - 1] * cnt % q;
                }

                f_inv[n] = qmi(f[n],q - q/p - 1);//phi(q) - 1
                for(int i = n;i >= 1;i--)
                {
                    int x = i;
                    while(x % p == 0)x /= p;
                    f_inv[i - 1] = f_inv[i] * x % q;
                }
            }
        }
        int F_c(int n)
        {
            int ans = 0;
            while(n)
            {
                ans += n / p;
```

```
            n /= p;
        }
        return ans;
    }
    int F_f(int n)
    {
        if(n == 0)return 1;
        int ans = qmi(s[q - 1],n / q) * s[n % q] % q * F_f(n / p) % q;
        return ans;
    }
    int F_f_inv(int n)
    {
        int ans = F_f(n);
        return qmi(ans,q - q / p - 1);
    }

    //n <= N,使用预处理，否则现算
    pii exlucas(int n,int m)
    {
        if(m > n)return {0,q};
        if(fl)
        {
            int cnt = c[n] - c[m] - c[n - m];
            int ans = qmi(p,cnt);
            int tmp = f[n] * f_inv[n - m] % q * f_inv[m] % q;
            return {ans * tmp % q,q};
        }
        else
        {
            int cnt = F_c(n) - F_c(m) - F_c(n - m);
            int ans = qmi(p,cnt);
            int tmp = F_f(n) * F_f_inv(n - m) % q * F_f_inv(m) % q;
            return {ans * tmp % q,q};
        }
    }
};
int M,siz;//M <= N;
std::vector<node>pri;//每个质因数预处理
//1 ~ n 范围组合数，对 M 取余
exLucas(int m,int n) : M(m),siz(0)
{
    while(m != 1)
    {
        int p = minp[m],k = 0,q = 1;
        while(minp[m] == p)
        {
            k++;
            q *= p;
            m /= p;
        }
        pri.push_back(node{p,k,q,n});
        siz++;
    }
```

```
    }
    int C(int n,int m)
    {
        if(m > n)return 0;
        std::vector<int>a(siz),b(siz);
        for(int i = 0;i < siz;i++)
        {
            pii ans = pri[i].exlucas(n,m);
            a[i] = ans.first;
            b[i] = ans.second;
        }
        return CRT::CRT(siz,a,b).first;
    }
};
```

数论分块

```
int k,l = 1,r,n;
//向下分块(l~r,tmp相等)
while(l <= n)
{
    tmp = n / l;
    r = n / tmp;
    k = tmp;
    l = r + 1;
}
//向上分块(l ~ r,k相等)
while(l <= n)
{
    tmp = (n - 1) / l;
    r = (n - 1) / tmp;//注意分母为0的情况(l == n)
    k = (n + l - 1) / l;

    l = r + 1;
}
//n维数论分块
```

线性求逆元

```
int n = 1e7;
vector<int>inv(n + 1);
for(int i = 2;i <= n;i++)inv[i] = (mod - mod / i) * inv[mod % i] % mod;
```

组合数

```
//普通预处理(<=2000)
constexpr int N = 2010;
constexpr int P = 1e9+7;
```

```cpp
int C[N][N];
void init()
{
    for(int i = 0; i < N;i++)
        for(int j = 0; j <= i;j++)
            if(!j)C[i][j] = 1;
            else C[i][j] = (C[i - 1][j] + C[i - 1][j - 1]) % P;
}

//预处理阶乘+逆元(<=1e5)
constexpr int N = 2e5 + 7;
constexpr int P = 998244353;
int fa[N],infa[N];
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % P;
        a = a * a % P;
        b >>= 1;
    }
    return res;
}
void init()
{
    fa[0] = 1;
    for(int i = 1;i < N;i++)fa[i] = fa[i - 1] * i % P;
    infa[N - 1] = qmi(fa[N - 1],P - 2);
    for(int i = N - 1;i >= 1;i--)infa[i - 1] = infa[i] * i % P;
}
int C(int a,int b)
{
    if(b > a)return 0;
    return fa[a] * infa[b] % P * infa[a - b] % P;
}

//卢卡斯定理(a,b<=1e18,询问次数<=20,模数p<=1e5)
int qmi(int a, int k, int p) {
    int res = 1 % p;
    while (k) {
        if (k & 1) res = res * a % p;
        a = a * a % p;
        k >>= 1;
    }
    return res;
}
// 定义求解
int C(int a, int b, int p) {
    if (b > a) return 0;
    int res = 1;
    for (int i = 1, j = a; i <= b; ++i, --j) {
        res = res * j % p;
```

```
        res = res * qmi(i, p - 2, p) % p;
    }
    return res;
}

int lucas(int a, int b, int p) {
    if (a < p && b < p) return C(a, b, p);
    return C(a % p, b % p, p) * lucas(a / p, b / p, p) % p;
}
void print(int a,int b,int c)
{
    lucas(a, b, p);
}
```

三维前缀和

```
//容斥原理
for (int i = 0; i <= 150; i++)
{
    for (int j = 0; j <= 150; j++)
    {
        for (int k = 0; k <= 150; k++)
        {
            if(j-1>=0)ans[i][j][k] += ans[i][j - 1][k]; //后方
            if(i-1>=0)ans[i][j][k] += ans[i - 1][j][k];//左侧
            if(k-1>=0)ans[i][j][k] += ans[i][j][k - 1]; //下方
            if (i - 1 >= 0 && j - 1 >= 0)ans[i][j][k] -= ans[i - 1][j - 1][k] - mod;//左后
            if (i - 1 >= 0 && k - 1 >= 0)ans[i][j][k] -= ans[i - 1][j][k - 1]-mod; //左下
            if (k - 1 >= 0 && j - 1 >= 0)ans[i][j][k] -= ans[i][j - 1][k - 1]-mod; //后下
            if (i - 1 >= 0 && j - 1 >= 0&&k-1>=0)ans[i][j][k] += ans[i - 1][j - 1][k - 1];
            ans[i][j][k] %= mod;
        }
    }
}
//查询
int q;cin >> q;
while(q--)
{
    int lx,rx,ly,ry,lz,rz;
    cin >> lx >> rx >> ly >> ry >> lz >> rz;
    int sum = ans[rx][ry][rz];
    lx--,ly--,lz--;
    sum -= ans[rx][ry][lz] + ans[rx][ly][rz] + ans[lx][ry][rz];
    sum += ans[rx][ly][lz] + ans[lx][ry][lz] + ans[lx][ly][rz];
    sum -= ans[lx][ly][lz];
    cout << sum << endl;
}
```

线性筛

```cpp
//筛素数
std::vector<int>pri;
bool vis[N];
int minp[N];//存数的最小质因子
void init()
{
    for(int i = 2;i < N;i++)
    {
        if(!vis[i]){pri.push_back(i);minp[i] = i;}
        for(auto x : pri)
        {
            if(i * x >= N)break;
            vis[i * x] = 1;
            minp[i * x] = x;
            if(i % x == 0)break;
        }
    }
}
//线性筛 求欧拉函数
vector<int>pri;
bool vis[N];
int phi[N];
void init()
{
    phi[1] = 1;
    for(int i = 2;i < N;i++)
    {
        if(!vis[i])
        {
            pri.push_back(i);
            phi[i] = i - 1;
        }
        for(auto x:pri)
        {
            if(x * i >= N)break;
            vis[i * x] = 1;
            if(i % x == 0)
            {
                phi[i * x] = x * phi[i];
                break;
            }
            else phi[i * x] = phi[i] * phi[x];
        }
    }
}
//线性筛 求莫比乌斯函数
vector<int>pri;
bool vis[N];
int mu[N];
void init()
```

```
{
    mu[1] = 1;
    for(int i = 2;i < N;i++)
    {
        if(!vis[i])
        {
            mu[i] = -1;
            pri.push_back(i);
        }
        for(int v : pri)
        {
            if(i * v >= N)break;
            vis[i * v] = 1;
            if(i % v == 0)
            {
                mu[i * v] = 0;
                break;
            }
            mu[i * v] = -mu[i];
        }
    }
}
```

欧拉函数

```
//phi(n) = n * (pi - 1) / pi;
//pi 为 n 的所有质因数集合
int euler_phi(int n)
{
    int ans = n;
    for(int i = 2;i * i <= n;i++)
        if (n % i == 0)
        {
            ans = ans / i * (i - 1);
            while(n % i == 0)n /= i;
        }
    if(n > 1)ans = ans / n * (n - 1);
    return ans;
}
```

求一个数的最大质因子(1e18)

```
int mul(int a,int b,int m)
{
    ull c = (ull)a * (ull)b - (ull)((long double)a / m * b + 0.5L) * (ull)m;
    if(c < (ull)m)return c;
    return c + m;
}
int qmi(int a,int b,int p)
{
    int res = 1;
```

```
    while(b)
    {
        if(b & 1)res = mul(res,a,p);
        b >>= 1;
        a = mul(a,a,p);
    }
    return res;
}
bool Miller_Rabin(int p)//判是否是素数
{
    if(p < 2)return false;
    if(p == 2 || p == 3)return true;
    int d = p - 1,r = 0;
    while(!(d & 1))r++,d >>= 1;  // 将d处理为奇数
    for(int k = 0;k < 10;k++)
    {
        int a = rand() % (p - 2) + 2;
        int x = qmi(a, d, p);
        if (x == 1 || x == p - 1) continue;
        for(int i = 0;i < r - 1;i++)
        {
            x = mul(x,x,p);
            if(x == p - 1)break;
        }
        if (x != p - 1) return false;
    }
    return true;
}
int Pollard_Rho(int x)//取x的一个因数
{
    int s = 0, t = 0;
    int c = (int)rand() % (x - 1) + 1;
    int step = 0,goal = 1;
    int val = 1;
    for(goal = 1;;goal <<= 1,s = t,val = 1)
    {
        for(step = 1;step <= goal;step++)
        {

            t = (mul(t,t,x) + c) % x;
            val = mul(val,llabs(t - s),x);
            if(step % 127 == 0)
            {
                int d = gcd(val,x);
                if(d > 1)return d;
            }
        }
        int d = gcd(val,x);
        if(d > 1)return d;
    }
}
int mx;
void fac(int x)
```

```
{
    if(x <= mx || x < 2)return;
    if(Miller_Rabin(x))
    {
        mx = max(mx,x);
        return;
    }
    int p = x;
    while(p >= x)p = Pollard_Rho(x);
    while(x % p == 0)x /= p;
    fac(x), fac(p);  // 继续向下分解x和p
}
void solve()
{
    srand((unsigned)time(NULL));
    int n;
    cin >> n;
    mx = 0;
    fac(n);
    if(mx == n)cout << "Prime" << endl;
    else cout << mx << endl;
}
```

卡特兰数

```
int Ca(int n)
{
    if(n<=1) return 1;
    vector<int>f(n + 1);
    f[0] = f[1] = 1;
    for(int i = 1;i <= n;i++)
    {
        f[i] = 0;
        for(int j = 0;j < i;j++)
            f[i] += (f[i] * f[i - j - 1]);
    }
    return f[n];
}
//Ca[n] = Ca[n - 1] * (4 * n - 2) / (n + 1);
//Ca[n] = C(2n,n) - C(2n,n - 1);
//Ca[n] = Ca[i] * Ca[n - i - 1] (i,1,n);
```

快速傅里叶变换(FFT)

```
typedef complex<double> cp;
const double pi = acos(-1);
vector<int>rev;//位逆序置换
void fft(vector<cp> &a,int flag = 1)//1为DFT,-1为IDFT
{
```

```
    int n = a.size();
    if(rev.size() != n)
    {
        rev.resize(n);
        int k = __builtin_ctz(n) - 1;
        for(int i = 0;i < n;i++)
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
    }
    for(int i = 0;i < n;i++)
        if(rev[i] < i)swap(a[i],a[rev[i]]);
    for(int len = 1;len < n;len *= 2)
    {
        cp wn = exp(cp(0,flag * pi / len));//单位根
        for(int i = 0;i < n;i += 2 * len)//合并每一组
        {
            cp w = cp(1,0);//w0
            for(int j = 0;j < len;j++)//合并
            {
                cp x = a[i + j];
                cp y = a[i + j + len] * w;
                a[i + j] = x + y;
                a[i + j + len] = x - y;
                w *= wn;//wi + 1;
            }
        }
    }
    if(flag == -1)
        for(int i = 0;i < n;i++)
            a[i] /= n;
}
vector<cp> mul(vector<cp> a,vector<cp> b)//求卷积
{
    int n = 1,tot = a.size() + b.size() - 1;
    while(n < tot)n <<= 1;
    a.resize(n);
    b.resize(n);
    fft(a);
    fft(b);
    for(int i = 0;i < n;i++)a[i] = a[i] * b[i];
    fft(a,-1);
    a.resize(tot);//x ^ (tot - 1)
    return a;
}

//应用1:高精度乘法
void solve()
{
    string s1,s2;
    cin >> s1 >> s2;
    int len1 = s1.size();
    int len2 = s2.size();
    vector<cp> a(len1),b(len2);
    for(int i = 0;i < len1;i++)a[i] = (double)(s1[len1 - 1 - i] - '0');
```

```
    for(int i = 0;i < len2;i++)b[i] = (double)(s2[len2 - 1 - i] - '0');
    a = mul(a,b);
    int len = a.size();
    vector<int>ans(len + 1);//多开1位存进位
    for(int i = 0;i < len;i++)
    {
        ans[i] += (int)(a[i].real() + 0.5);
        ans[i + 1] += ans[i] / 10;
        ans[i] %= 10;
    }
    while(len > 0 && !ans[len])len--;
    for(;len >= 0;len--)cout << ans[len];
    cout << endl;
}
```

快速数论变换(NTT)

```
const int mod = 998244353;
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % mod;
        b >>= 1;
        a = a * a % mod;
    }
    return res;
}
vector<int>rev;
void ntt(vector<int> &a,int flag = 1)
{
    int n = a.size();
    if(rev.size() != n)
    {
        rev.resize(n);
        int k = __builtin_ctz(n) - 1;
        for(int i = 0;i < n;i++)
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
    }
    for(int i = 0;i < n;i++)
        if(rev[i] < i)swap(a[i],a[rev[i]]);
    for(int len = 1;len < n;len *= 2)
    {
        int gn = qmi(3,(mod - 1) / (len << 1));//原根
        for(int i = 0;i < n;i += 2 * len)
        {
            int g = 1;
            for(int j = 0;j < len;j++)
            {
                int x = a[i + j];
                int y = a[i + j + len] * g % mod;
```

```
                a[i + j] = (x + y) % mod;
                a[i + j + len] = (x - y + mod) % mod;
                g = g * gn % mod;
            }
        }
    }
    if(flag == -1)
    {
        reverse(a.begin() + 1,a.end());//多个交换
        int inv = qmi(n,mod - 2);
        for(int i = 0;i < n;i++)
            a[i] = a[i] * inv % mod;
    }
}
vector<int> mul(vector<int> a,vector<int> b)
{
    int n = 1,tot = a.size() + b.size() - 1;
    while(n < tot) n <<= 1;
    a.resize(n);
    b.resize(n);
    ntt(a);
    ntt(b);
    for(int i = 0;i < n;i++)
        a[i] = a[i] * b[i] % mod;
    ntt(a,-1);
    a.resize(tot);
    return a;
}

//高精乘法
void solve()
{
    string s1,s2;
    cin >> s1 >> s2;
    int len1 = s1.size();
    int len2 = s2.size();
    vector<int>a(len1),b(len2);
    for(int i = 0;i < len1;i++)a[i] = s1[len1 - 1 - i] - '0';
    for(int i = 0;i < len2;i++)b[i] = s2[len2 - 1 - i] - '0';
    a = mul(a,b);
    int len = a.size();
    vector<int>ans(len + 1);
    for(int i = 0;i < len;i++)
    {
        ans[i] += a[i];
        ans[i + 1] += ans[i] / 10;
        ans[i] %= 10;
    }
    while(len > 0 && !ans[len])len--;
    for(;len >= 0;len--)cout << ans[len];
    cout << endl;
}
```

快速沃尔什变换(FWT)

```
int qmi(int a,int b)
{
    int res = 1;
    while(b)
    {
        if(b & 1)res = res * a % mod;
        b >>= 1;
        a = a * a % mod;
    }
    return res;
}
int add(int u,int v)
{
    return (u + v + mod) % mod;
}
int mul(int u,int v)
{
    return u * v % mod;
}
void fwt_or(vector<int>&a,int flag = 1)//-1为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
                a[i + j + len] = add(a[i + j + len],mul(flag,a[i + j]));
    return;
}
void fwt_and(vector<int>&a,int flag = 1)//-1为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
                a[i + j] = add(a[i + j],mul(flag,a[i + j + len]));
    return;
}
void fwt_xor(vector<int>&a,int flag = 1)//1/2为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
            {
                a[i + j] = add(a[i + j],a[i + j + len]);
                a[i + j + len] = add(a[i + j],-mul(2,a[i + j + len]));
                a[i + j] = mul(a[i + j],flag);
                a[i + j + len] = mul(a[i + j + len],flag);
            }
    return;
```

```cpp
}
void fwt_xnor(vector<int>&a,int flag = 1)//1/2为逆变换
{
    int n = a.size();
    for(int len = 1;len < n;len *= 2)
        for(int i = 0;i < n;i += len * 2)
            for(int j = 0;j < len;j++)
            {
                a[i + j + len] = add(a[i + j + len],a[i + j]);
                a[i + j] = add(a[i + j + len],-mul(2,a[i + j]));
                a[i + j + len] = mul(a[i + j + len],flag);
                a[i + j] = mul(a[i + j],flag);
            }
    return;
}
//应用：卷积运算
vector<int> work(vector<int>a,vector<int>b)
{
    int n = a.size();
    fwt(a);
    fwt(b);
    for(int i = 0;i < n;i++)a[i] = mul(a[i],b[i]);
    fwt(a,mod - 1);//or,and
    fwt(a,qmi(2,mod - 2))//xor,xnor
    return a;
}
```

多项式模板

```cpp
using u64 = unsigned long long;
using i64 =  long long;
using u32 = unsigned;

template<class T>
constexpr T power(T a, u64 b, T res = 1) {
    for (; b != 0; b /= 2, a *= a) {
        if (b & 1) {
            res *= a;
        }
    }
    return res;
}

template<u32 P>
struct ModIntBase {
public:
    constexpr ModIntBase() : x(0) {}
    template<class T>
    constexpr ModIntBase(T x_) {
        T v = x_ % T(mod());
        if (v < 0) {
            v += mod();
```

```
        }
        x = v;
    }

    constexpr static u32 mod() {
        return P;
    }

    constexpr u32 val() const {
        return x;
    }

    constexpr u32 mulMod(u32 a, u32 b) {
        return u64(a) * b % mod();
    }

    constexpr ModIntBase operator-() const {
        ModIntBase res;
        res.x = (x == 0 ? 0 : mod() - x);
        return res;
    }

    constexpr ModIntBase inv() const {
        return power(*this, mod() - 2);
    }

    constexpr ModIntBase &operator*=(const ModIntBase &rhs) & {
        x = mulMod(x, rhs.val());
        return *this;
    }
    constexpr ModIntBase &operator+=(const ModIntBase &rhs) & {
        x += rhs.val();
        if (x >= mod()) {
            x -= mod();
        }
        return *this;
    }
    constexpr ModIntBase &operator-=(const ModIntBase &rhs) & {
        x -= rhs.val();
        if (x >= mod()) {
            x += mod();
        }
        return *this;
    }
    constexpr ModIntBase &operator/=(const ModIntBase &rhs) & {
        return *this *= rhs.inv();
    }

    friend constexpr ModIntBase operator*(ModIntBase lhs, const ModIntBase &rhs) {
        lhs *= rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator+(ModIntBase lhs, const ModIntBase &rhs) {
```

```
        lhs += rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator-(ModIntBase lhs, const ModIntBase &rhs) {
        lhs -= rhs;
        return lhs;
    }
    friend constexpr ModIntBase operator/(ModIntBase lhs, const ModIntBase &rhs) {
        lhs /= rhs;
        return lhs;
    }

    friend constexpr std::istream &operator>>(std::istream &is, ModIntBase &a) {
        i64 i;
        is >> i;
        a = i;
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const ModIntBase &a) {
        return os << a.val();
    }

    friend constexpr bool operator==(const ModIntBase &lhs, const ModIntBase &rhs) {
        return lhs.val() == rhs.val();
    }

private:
    u32 x;
};

template<u32 P>
using ModInt = ModIntBase<P>;

//多项式部分,以上为取模板子
constexpr int P = 998244353;
using Z = ModInt<P>;
std::vector<int> rev;
std::vector<Z> roots{0, 1};
void dft(std::vector<Z> &a) {
    int n = a.size();

    if (int(rev.size()) != n) {
        int k = __builtin_ctz(n) - 1;
        rev.resize(n);
        for (int i = 0; i < n; i++) {
            rev[i] = rev[i >> 1] >> 1 | (i & 1) << k;
        }
    }

    for (int i = 0; i < n; i++) {
        if (rev[i] < i) {
            std::swap(a[i], a[rev[i]]);
        }
```

```
    }
    if (int(roots.size()) < n) {
        int k = __builtin_ctz(roots.size());
        roots.resize(n);
        while ((1 << k) < n) {
            Z e = power(Z(3), (P - 1) >> (k + 1));
            for (int i = 1 << (k - 1); i < (1 << k); i++) {
                roots[2 * i] = roots[i];
                roots[2 * i + 1] = roots[i] * e;
            }
            k++;
        }
    }
    for (int k = 1; k < n; k *= 2) {
        for (int i = 0; i < n; i += 2 * k) {
            for (int j = 0; j < k; j++) {
                Z u = a[i + j];
                Z v = a[i + j + k] * roots[k + j];
                a[i + j] = u + v;
                a[i + j + k] = u - v;
            }
        }
    }
}
void idft(std::vector<Z> &a) {
    int n = a.size();
    std::reverse(a.begin() + 1, a.end());
    dft(a);
    Z inv = (1 - P) / n;
    for (int i = 0; i < n; i++) {
        a[i] *= inv;
    }
}
struct Poly {
    std::vector<Z> a;
    Poly() {}
    explicit Poly(int size, std::function<Z(int)> f = [](int) { return 0; }) : a(size)
{
        for (int i = 0; i < size; i++) {
            a[i] = f(i);
        }
    }
    Poly(const std::vector<Z> &a) : a(a) {}
    Poly(const std::initializer_list<Z> &a) : a(a) {}
    int size() const {
        return a.size();
    }
    void resize(int n) {
        a.resize(n);
    }
    Z operator[](int idx) const {
        if (idx < size()) {
            return a[idx];
```

```
        } else {
            return 0;
        }
    }
    Z &operator[](int idx) {
        return a[idx];
    }
    Poly mulxk(int k) const {
        auto b = a;
        b.insert(b.begin(), k, 0);
        return Poly(b);
    }
    Poly modxk(int k) const {
        k = std::min(k, size());
        return Poly(std::vector<Z>(a.begin(), a.begin() + k));
    }
    Poly divxk(int k) const {
        if (size() <= k) {
            return Poly();
        }
        return Poly(std::vector<Z>(a.begin() + k, a.end()));
    }
    friend Poly operator+(const Poly &a, const Poly &b) {
        std::vector<Z> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = a[i] + b[i];
        }
        return Poly(res);
    }
    friend Poly operator-(const Poly &a, const Poly &b) {
        std::vector<Z> res(std::max(a.size(), b.size()));
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = a[i] - b[i];
        }
        return Poly(res);
    }
    friend Poly operator-(const Poly &a) {
        std::vector<Z> res(a.size());
        for (int i = 0; i < int(res.size()); i++) {
            res[i] = -a[i];
        }
        return Poly(res);
    }
    friend Poly operator*(Poly a, Poly b) {
        if (a.size() == 0 || b.size() == 0) {
            return Poly();
        }
        if (a.size() < b.size()) {
            std::swap(a, b);
        }
        if (b.size() < 128) {
            Poly c(a.size() + b.size() - 1);
            for (int i = 0; i < a.size(); i++) {
```

```
                for (int j = 0; j < b.size(); j++) {
                    c[i + j] += a[i] * b[j];
                }
            }
            return c;
        }
        int sz = 1, tot = a.size() + b.size() - 1;
        while (sz < tot) {
            sz *= 2;
        }
        a.a.resize(sz);
        b.a.resize(sz);
        dft(a.a);
        dft(b.a);
        for (int i = 0; i < sz; ++i) {
            a.a[i] = a[i] * b[i];
        }
        idft(a.a);
        a.resize(tot);
        return a;
    }
    friend Poly operator*(Z a, Poly b) {
        for (int i = 0; i < int(b.size()); i++) {
            b[i] *= a;
        }
        return b;
    }
    friend Poly operator*(Poly a, Z b) {
        for (int i = 0; i < int(a.size()); i++) {
            a[i] *= b;
        }
        return a;
    }
    Poly &operator+=(Poly b) {
        return (*this) = (*this) + b;
    }
    Poly &operator-=(Poly b) {
        return (*this) = (*this) - b;
    }
    Poly &operator*=(Poly b) {
        return (*this) = (*this) * b;
    }
    Poly &operator*=(Z b) {
        return (*this) = (*this) * b;
    }
    Poly deriv() const {
        if (a.empty()) {
            return Poly();
        }
        std::vector<Z> res(size() - 1);
        for (int i = 0; i < size() - 1; ++i) {
            res[i] = (i + 1) * a[i + 1];
        }
```

```
        return Poly(res);
    }
    Poly integr() const {
        std::vector<Z> res(size() + 1);
        for (int i = 0; i < size(); ++i) {
            res[i + 1] = a[i] / (i + 1);
        }
        return Poly(res);
    }
    Poly inv(int m) const {
        Poly x{a[0].inv()};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x * (Poly{2} - modxk(k) * x)).modxk(k);
        }
        return x.modxk(m);
    }
    Poly log(int m) const {
        return (deriv() * inv(m)).integr().modxk(m);
    }
    Poly exp(int m) const {
        Poly x{1};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x * (Poly{1} - x.log(k) + modxk(k))).modxk(k);
        }
        return x.modxk(m);
    }
    Poly pow(int k, int m) const {
        int i = 0;
        while (i < size() && a[i].val() == 0) {
            i++;
        }
        if (i == size() || 1LL * i * k >= m) {
            return Poly(std::vector<Z>(m));
        }
        Z v = a[i];
        auto f = divxk(i) * v.inv();
        return (f.log(m - i * k) * k).exp(m - i * k).mulxk(i * k) * power(v, k);
    }
    Poly sqrt(int m) const {
        Poly x{1};
        int k = 1;
        while (k < m) {
            k *= 2;
            x = (x + (modxk(k) * x.inv(k)).modxk(k)) * ((P + 1) / 2);
        }
        return x.modxk(m);
    }
    Poly mulT(Poly b) const {
        if (b.size() == 0) {
```

```
            return Poly();
        }
        int n = b.size();
        std::reverse(b.a.begin(), b.a.end());
        return ((*this) * b).divxk(n - 1);
    }
    std::vector<Z> eval(std::vector<Z> x) const {
        if (size() == 0) {
            return std::vector<Z>(x.size(), 0);
        }
        const int n = std::max(int(x.size()), size());
        std::vector<Poly> q(4 * n);
        std::vector<Z> ans(x.size());
        x.resize(n);
        std::function<void(int, int, int)> build = [&](int p, int l, int r) {
            if (r - l == 1) {
                q[p] = Poly{1, -x[l]};
            } else {
                int m = (l + r) / 2;
                build(2 * p, l, m);
                build(2 * p + 1, m, r);
                q[p] = q[2 * p] * q[2 * p + 1];
            }
        };
        build(1, 0, n);
        std::function<void(int, int, int, const Poly &)> work = [&](int p, int l, int
r, const Poly &num) {
            if (r - l == 1) {
                if (l < int(ans.size())) {
                    ans[l] = num[0];
                }
            } else {
                int m = (l + r) / 2;
                work(2 * p, l, m, num.mulT(q[2 * p + 1]).modxk(m - l));
                work(2 * p + 1, m, r, num.mulT(q[2 * p]).modxk(r - m));
            }
        };
        work(1, 0, n, mulT(q[1].inv(n)));
        return ans;
    }
};
```

取模类(int)

```
template<class T>
constexpr T power(T a, i64 b) {
    T res = 1;
    while(b)
    {
        if(b & 1)res *= a;
        a *= a;
        b >>= 1;
```

```
    }
    return res;
}

template<int P>
struct MInt {
    int x;
    constexpr MInt() : x{} {}
    constexpr MInt(i64 x) : x{norm(x % getMod())} {}

    static int Mod;
    constexpr static int getMod() {
        if (P > 0) {
            return P;
        } else {
            return Mod;
        }
    }
    constexpr static void setMod(int Mod_) {
        Mod = Mod_;
    }
    constexpr int norm(int x) const {
        if (x < 0) {
            x += getMod();
        }
        if (x >= getMod()) {
            x -= getMod();
        }
        return x;
    }
    constexpr int val() const {
        return x;
    }
    explicit constexpr operator int() const {
        return x;
    }
    constexpr MInt operator-() const {
        MInt res;
        res.x = norm(getMod() - x);
        return res;
    }
    constexpr MInt inv() const {
        assert(x != 0);
        return power(*this, getMod() - 2);
    }
    constexpr MInt &operator*=(MInt rhs) & {
        x = 1LL * x * rhs.x % getMod();
        return *this;
    }
    constexpr MInt &operator+=(MInt rhs) & {
        x = norm(x + rhs.x);
        return *this;
    }
```

```
    constexpr MInt &operator-=(MInt rhs) & {
        x = norm(x - rhs.x);
        return *this;
    }
    constexpr MInt &operator/=(MInt rhs) & {
        return *this *= rhs.inv();
    }
    friend constexpr MInt operator*(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res *= rhs;
        return res;
    }
    friend constexpr MInt operator+(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res += rhs;
        return res;
    }
    friend constexpr MInt operator-(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res -= rhs;
        return res;
    }
    friend constexpr MInt operator/(MInt lhs, MInt rhs) {
        MInt res = lhs;
        res /= rhs;
        return res;
    }
    friend constexpr std::istream &operator>>(std::istream &is, MInt &a) {
        i64 v;
        is >> v;
        a = MInt(v);
        return is;
    }
    friend constexpr std::ostream &operator<<(std::ostream &os, const MInt &a) {
        return os << a.val();
    }
    friend constexpr bool operator==(MInt lhs, MInt rhs) {
        return lhs.val() == rhs.val();
    }
    friend constexpr bool operator!=(MInt lhs, MInt rhs) {
        return lhs.val() != rhs.val();
    }
};

template<>
int MInt<0>::Mod = 998244353;

//常数逆元
template<int V, int P>
constexpr MInt<P> CInv = MInt<P>(V).inv();

constexpr int P = 1000000007;//使用模数
using Z = MInt<P>;
```